SKILLSPASSAURORA

Software Requirements Specification

A detailed specification of the skills-first employment marketplace, workflows, system requirements, security controls, and data model.

VERSION 3.0

SkillsPassAURORA
Proving What You Can Do

Document status: Baseline specification of the implemented system
Prepared: 22 October 2026
System: SkillsPassAurora
Application type: PHP web application with MySQL database

DOCUMENT CONTROL

Specification overview

This SRS describes the system as implemented in the project at the time of preparation. It is intended to be used as a requirements baseline for operation, maintenance, testing, and future enhancements.

Document	SkillsPassAurora Software Requirements Specification
Version	3.0
Date	22 October 2026
System boundary	SkillsPassAurora web application, PHP runtime, MySQL database, and configured SMTP delivery for sign-in OTP messages.
Audience	Project owners, developers, system administrators, testers, employers, and candidate/employee users.
Specification basis	Project PHP source, database schema, and setup documentation. This document records existing behavior and does not assert unimplemented capabilities.

Revision history

Version	Date	Summary
3.0	22 October 2026	Detailed SRS baseline with user workflows, security requirements, database entities, and ER diagram.

Contents

1. Purpose, scope, and system context
2. Users, operating environment, and constraints
3. End-to-end workflows and use cases
4. Functional requirements and business rules
5. Data model and entity-relationship diagram
6. External interfaces and non-functional requirements
7. Security, privacy, error handling, and acceptance criteria

> **Terminology:** The database and authorization role is *candidate*; parts of the user interface call the same job-seeking user an *employee*. The terms refer to the same account type in this specification.

1 · SYSTEM CONTEXT

Purpose and scope

1.1 Purpose

SkillsPassAurora is a two-sided hiring marketplace that lets candidates present their experience and apply for work while employers publish job opportunities and review applications. The system supports a skills-first process: employers can read a candidate's cover letter, relevant experience, and optional portfolio link before deciding whom to select.

1.2 Product scope

The application provides public job discovery, candidate and employer account registration, password authentication, conditional email OTP verification after repeated password failures, employer job posting, candidate applications, employer applicant review and selection, and job availability status. The landing page presents the service, a four-second photo slideshow, and links to browse jobs and account flows.

1.3 System boundary

Inside the system	External dependency / outside boundary
Web routing and HTML rendering; PHP request processing; session state; validation; authorization checks; database operations; OTP generation and verification; job/application state transitions.	Browser and user device; Apache or another PHP-capable web server; PHP 8.1+; MySQL 8+; PHPMailer dependency; SMTP provider and recipient mail system; operating-system configuration and backups.

1.4 Product perspective

The application is a server-rendered PHP site. The principal request controller and page templates are implemented in `index.php`, database access/session helpers are implemented in `config.php`, persistent schema is defined in `schema.sql`, and visual presentation is provided by `style.css`. MySQL is the system of record for users, jobs, and applications. Authentication-pending OTP state is kept in the database and tied to a PHP session.

1.5 Out of scope

- Third-party job-board synchronization, payments, subscriptions, interviews, or calendar scheduling.
- Resume/CV uploads, in-app messaging, application withdrawal/editing, and candidate profile editing.
- Employer job editing/deletion and automated matching or ranking.
- Password reset, account recovery, identity proofing, email registration verification, and administrator moderation.
- Guaranteed delivery of email outside the configured SMTP provider.

Users, environment, and constraints

2.1 User classes

User class	Goals	Permitted capabilities
Visitor	Understand the service and inspect job opportunities.	View landing page and job listings. Must sign in as a candidate to apply.
Candidate / employee	Discover suitable jobs and submit an application.	Register as candidate, sign in, browse jobs, submit one application per job, and view own application status.
Employer	Advertise a role and select a suitable applicant.	Register as employer, sign in, post jobs, view applications for own jobs, and select a pending applicant for an available job.
System operator	Install and maintain the service.	Configure PHP/database/SMTP environment, import schema, manage server operation and logs. No in-app administrator console is implemented.

2.2 Operating environment

- PHP 8.1 or later with PDO MySQL enabled.
- MySQL 8 or later with InnoDB support and UTF-8 capable `utf8mb4` collation.
- Composer-installed PHPMailer dependency.
- Modern browser with HTML, CSS, JavaScript, cookies, and network access to the application.
- SMTP host, port, username, password/app password, and sender configured for email OTP delivery.

2.3 Constraints and assumptions

- One email address identifies one account; account role is either candidate or employer.
- Registration requires a valid email, display name/company name, role, and password of at least 10 characters. Email verification is not required at registration.
- All timestamps rely on application/database clocks; OTP comparisons use UTC database timestamps and PHP time.
- SMTP is an external dependency only for OTP messages; ordinary registration and sign-in below the OTP threshold do not require email delivery.
- Database availability is required to load job listings and perform account, posting, application, and hiring actions.

2.4 Navigation and page routes

Route	Purpose	Access / behavior
`index.php`	Public landing page.	Anonymous visitor; authenticated users normally see their dashboard.
`index.php?page=jobs`	Dedicated available-jobs discovery page.	Public listing; signed-in candidates can apply; signed-in employers can inspect their own applicants.
`index.php?page=auth&mode=login`	Sign in.	Public; pending OTP challenge can supersede the login view.
`index.php?page=auth&mode=signup`	Choose account type and register.	Public; optional `role=candidate` or `role=employer` opens that role's registration form.
Server-managed dashboard and verification views	Employer/candidate dashboard and sign-in OTP challenge.	Dashboard requires an authenticated session; OTP view requires a pending login challenge.

End-to-end workflows

3.1 Registration and session creation

1. **Select role:** Visitor opens Get started, then chooses Employee/Candidate or Employer. Role is included in the form and checked again by the server.
2. **Enter details:** User supplies name/company, email, and password. The server normalizes email to lowercase, validates its format, requires a name and supported role, and enforces a 10-character minimum password.
3. **Persist account:** Password is stored as a password hash. A unique email constraint prevents duplicate accounts. Registration does not send a verification email.
4. **Start session:** After database commit, the session ID is regenerated and the account ID, email, role, and display name are saved in the session. User is taken to the dashboard.

3.2 Sign-in, failed-attempt threshold, and OTP

1. **Verify password:** User supplies email and password. The system loads the matching account under a database row lock and verifies the stored password hash.
2. **Failed password:** For an existing account, the failed counter is increased but capped at 10. A generic credential error is returned. Unknown email addresses do not have a counter to update.
3. **Successful password below threshold:** If fewer than 10 failures are recorded, the counter resets, the session ID is regenerated, and the user is signed in.
4. **Successful password at/above threshold:** The system creates a random six-digit OTP, stores only its password hash with a 10-minute expiry, resets the OTP attempt counter, and sends it through SMTP. The user is placed in a pending-login session and shown the verification page. If a still-valid OTP exists, the system reuses the challenge instead of sending another code immediately.
5. **Verify OTP:** User enters the code. The system checks challenge presence, expiry, hash, and the maximum of five attempts. Success clears OTP fields, resets failed-password attempts, regenerates the session ID, and opens the dashboard.
6. **Resend:** User may request a new code after the 60-second resend interval. Resending replaces the hash, sets a fresh 10-minute expiry, and resets OTP attempts. SMTP failure is reported; the code is not shown on screen.

> **Important boundary:** The threshold is per existing user account, not a global IP/device lockout. Unknown-account attempts are not recorded in the user table.

3.3 Logout

The user submits the CSRF-protected sign-out form. The application clears session data, regenerates the session identifier, and displays the public landing page. Responses use no-store/no-cache headers to reduce the chance of private pages being restored from browser caches.

Employer and candidate workflows

3.4 Employer job posting

1. Employer signs in or creates an employer account and opens the dashboard.
2. Employer enters a title (required, up to 150 characters), location (required, up to 150), employment type, description (required, up to 10,000 characters), and optional skills (up to 2,000).
3. Server requires a signed-in employer, validates all values and supported employment type, and confirms the employer account in the database.
4. Job is inserted with `available` status and linked to its employer. It becomes visible in public job browsing; available listings sort before taken listings, then by creation time.

3.5 Candidate job discovery and application

1. Visitor opens Browse jobs. Each listing presents employer name, title, location, employment type, description, optional requested skills, application count, and Available/Taken status.
2. Anonymous visitors are prompted to sign in to apply. A candidate can apply only to an available job.
3. Candidate completes a required cover letter (1–5,000 characters), optional relevant experience (up to 3,000), and optional valid HTTP/HTTPS portfolio URL (up to 2,048 characters).
4. Server validates candidate role, inputs, and current job availability. A transaction locks the job while checking availability and inserts the application.
5. A unique database key prevents a second application by the same candidate for the same job. Candidate sees the application status on subsequent listings/dashboard loads.

3.6 Employer review and selection

1. Employer views application details only within a job they own. The page presents candidate name/email, cover letter, optional experience, optional portfolio link, and application status.
2. For a pending application on an available owned job, employer selects the candidate.
3. Server checks employer role and locks the owned job. It verifies that the selected candidate has a pending application for that job.
4. Within a database transaction the selected application becomes Accepted, the job becomes Taken with the selected candidate ID, and remaining pending applications for that job become Rejected.
5. If any required write fails, the transaction is rolled back. Taken jobs no longer accept applications.

3.7 Job and application states

Record	Initial state	Transition	Result
Job	Available	Employer selects an eligible pending applicant.	Taken; selected candidate is recorded.
Application	Pending	Selected by employer.	Accepted.
Other pending applications for selected job	Pending	Another applicant is accepted.	Rejected.
Sign-in OTP challenge	Absent or expired	Correct password after 10 or more failed attempts.	Pending challenge with hash, expiry and attempt count; challenge is cleared on successful verification.

Functional requirements

Requirements use “shall” to describe observable system behavior. Priority is marked **Core** for necessary marketplace operation and **Supporting** for ancillary interaction.

ID	Requirement	Priority
FR-01	The system shall display a public landing page with product identity, job-discovery and account links, opportunity categories, employer call-to-action, and a rotating photo slideshow. The slideshow interval shall be four seconds unless user interaction, hover/focus pause, or reduced-motion preference pauses automatic rotation.	Supporting
FR-02	The system shall provide a dedicated public jobs page at `index.php?page=jobs`, listing jobs with status, employer, title, location, employment type, description, optional skills, and application count.	Core
FR-03	The system shall permit account registration as exactly one of candidate or employer, requiring a valid email, name/company name, and password of at least 10 characters.	Core
FR-04	The system shall store password hashes and shall not require email verification during registration.	Core
FR-05	The system shall authenticate registered users and clear a user's failed-password counter after successful password-only sign-in when the counter is below 10.	Core
FR-06	The system shall increment an existing account's failed-password counter on an incorrect password, capped at 10. It shall require an email OTP after a correct password when at least 10 failures have accrued.	Core
FR-07	The OTP shall be a six-digit random value; only a hash shall be persisted. The challenge shall expire after 10 minutes, allow at most five verification attempts, and enforce a minimum 60-second resend interval.	Core
FR-08	The system shall allow authenticated employers to post jobs with validated title, location, supported employment type, description, and optional skills. New jobs shall be Available.	Core
FR-09	The system shall allow authenticated candidates to apply only to Available jobs, requiring a cover letter and accepting optional experience and an HTTP/HTTPS portfolio URL.	Core
FR-10	The system shall prevent duplicate applications by the same candidate for the same job and shall return a user-readable error if the database uniqueness rule is violated.	Core
FR-11	The system shall allow an employer to review applicant details only for that employer's own jobs.	Core
FR-12	The system shall allow an employer to select a pending applicant only for an Available job owned by that employer; it shall mark the chosen application Accepted, the job Taken, and other pending applications Rejected as one transaction.	Core
FR-13	The system shall show each candidate the status of their own application and shall show each employer applicant details only for jobs owned by that employer.	Core
FR-14	The system shall require CSRF-token validation on state-changing POST requests and shall provide explicit feedback for validation, database, and email-delivery failures.	Core
FR-15	The system shall clear session data on logout, regenerate session identifiers at authentication boundaries, and issue no-cache headers for sensitive responses.	Core
FR-16	The system shall offer consistent navigation between the public home page, jobs page, contact anchor, sign-in, and account creation views.	Supporting

4.1 Input rules and limits

Field	Rule
Email	Valid email syntax; normalized to lowercase at registration/sign-in; maximum stored length 190 characters; unique.
Password	Minimum 10 characters at registration; stored using PHP password hashing; validated with password verification.

Field	Rule
Job title / location	Required; up to 150 characters each.
Employment type	Exactly one of full-time, part-time, contract, temporary.
Job description / skills	Description required, up to 10,000 characters; skills optional, up to 2,000.
Application	Cover letter required, up to 5,000; relevant experience optional, up to 3,000; portfolio URL optional, valid HTTP/HTTPS and up to 2,048.

5 · DATA MODEL

Database entities and ER diagram

The MySQL database is named `aurora_group3`. All three business entities use InnoDB. Foreign keys enforce ownership and referential integrity; email and candidate/job application pairs are unique.

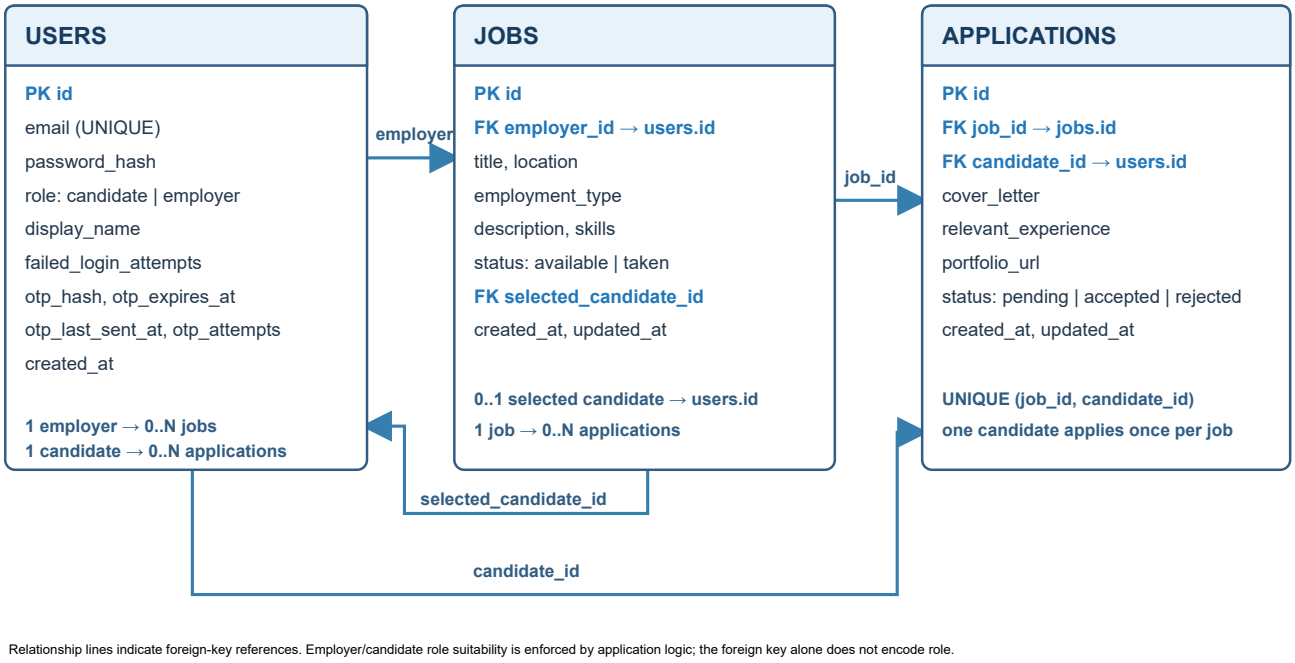


Figure 1. Entity relationships for users, jobs, and applications. The selected-candidate foreign key on jobs also references `users.id`.

5.1 Entity definitions

Entity	Primary key / important constraints	Purpose and lifecycle
users	`id`; unique `email`; enum `role`; password hash; failed-login/OTP fields.	Stores candidate and employer identities and the authentication security state. Deleting a user cascades to their employer jobs or candidate applications as defined by related foreign keys.
jobs	`id`; FK `employer_id`; optional FK `selected_candidate_id`; status enum; indexes for status/date and employer.	Stores employer opportunities. Employer deletion cascades to jobs; selected-candidate deletion sets the selected-candidate reference to null.
applications	`id`; FKs `job_id`, `candidate_id`; unique (`job_id`, `candidate_id`); status enum.	Stores one candidate's submission and decision for a job. Job or candidate deletion cascades to applications.

Interfaces and non-functional requirements

6.1 User interface

- Server-rendered responsive HTML/CSS pages shall support public, candidate, and employer workflows on desktop and mobile viewports.
- Navigation shall consistently provide home, job browsing, contact, sign-in, and account creation access; landing-page registration copy removed from hero while account signup remains available in navigation.
- Job cards shall show Available or Taken status. Forms shall label fields and provide inline constraints where practical.
- OTP slide controls and password visibility controls shall be keyboard-operable; slideshow shall honor reduced-motion preferences and pause while hovered or focused.
- Portfolio links are opened in a separate browsing context with `noopener noreferrer`.

6.2 Software interfaces

Interface	Requirements
PHP and MySQL	PDO connections use UTF-8, exception error mode, associative fetch mode, and native prepared statements.
SMTP	PHPMailer sends sign-in OTP messages using configured SMTP variables. Missing credentials or send errors are logged server-side and reported to the user without disclosing the OTP.
Browser	HTTP GET renders views; state-changing forms submit POST with session CSRF token. Session cookie is HttpOnly, SameSite=Lax, and Secure when HTTPS is active.

6.3 Non-functional requirements

ID	Quality attribute	Requirement
NFR-01	Security	Passwords and OTPs shall never be stored as plaintext. OTP hashes, expiry, maximum attempts, and resend throttling shall be enforced server-side.
NFR-02	Authorization	Candidate-only actions require candidate role; employer-only actions require employer role; applicant data and candidate selection are restricted to the employer who owns the job.
NFR-03	Data integrity	Foreign keys, uniqueness constraints, transactions, and row locks shall maintain ownership, prevent duplicate applications, and protect hiring state transitions.
NFR-04	Input/output safety	Database inputs shall use prepared statements. Dynamic HTML output shall be escaped using UTF-8-safe HTML escaping.
NFR-05	Privacy	Application details are private to the candidate and the owning employer. Sign-out clears server session data; authenticated responses are marked no-store/no-cache.
NFR-06	Reliability	Hiring changes shall commit together or roll back. Database and email failures shall not be presented as successful operations.
NFR-07	Compatibility	Deployment requires PHP 8.1+, PDO MySQL, MySQL 8+, Composer dependencies, and browser support for modern HTML/CSS/JavaScript.
NFR-08	Usability	Public browsing is available without an account. Validation, authentication, delivery, and availability failures shall be explained in user-readable language.
NFR-09	Accessibility	Forms have associated labels, controls have accessible names, slides expose active-state information, and the UI supports reduced-motion preference.
NFR-10	Maintainability	Database schema and deployment settings are documented; credentials and SMTP secrets are provided through environment configuration, not committed source.

6.4 Security and privacy controls

- CSRF token is generated with cryptographically secure randomness and compared with `hash_equals`; invalid form sessions are rejected.
- Session identifiers are regenerated when login, registration, and OTP verification establish a user session. Session cookies use HttpOnly and SameSite=Lax; Secure is conditional on HTTPS.
- OTP codes use cryptographic random generation and password hashing; expiration, five-try ceiling, and one-minute resend delay are checked on the server.
- Database queries use PDO prepared statements. Dynamic user-controlled text is escaped before HTML rendering.
- Employer ownership is checked server-side before applicant review or candidate selection. Portfolio URLs are restricted to HTTP/HTTPS.
- SMTP credentials are external configuration. Error logs contain transport context, not the OTP value.

Error handling, acceptance, and deployment

7.1 Error handling expectations

Condition	Expected system behavior
Invalid or duplicate registration	Reject submission, preserve no invalid account, and show validation or duplicate-email message.
Incorrect password	Show a generic credential error; increment capped failed counter for an existing account.
Expired/invalid OTP or fifth-attempt exhaustion	Reject verification, do not authenticate, and require a valid resend/new challenge as appropriate.
SMTP not configured or delivery fails	Log an error, report email delivery failure, never display the code to the browser.
Job unavailable or invalid application data	Reject application and show a validation or availability message.
Duplicate application	Database uniqueness constraint prevents duplicate; show duplicate-application feedback.
Unauthorized role or foreign employer job	Reject the state-changing action; do not disclose another employer's applicant data.
Database unavailable/schema mismatch	Show a clear setup or request failure message; log relevant database action and error metadata server-side.

7.2 Acceptance criteria

1. A visitor can load the landing page and public jobs page without signing in.
2. A visitor can register as either supported role with valid input; duplicate emails and passwords shorter than 10 characters are rejected.
3. A correct password signs in below the OTP threshold. Ten failed attempts followed by a correct password requires the email OTP challenge.
4. An OTP expires after 10 minutes, allows no more than five verification attempts, and cannot be resent more frequently than once per minute.
5. An employer can post a valid job; it appears as Available in public listings and is associated with that employer.
6. A candidate can submit one valid application to an Available job with a cover letter and optional experience/portfolio. A duplicate or Taken-job application is rejected.
7. An employer can read and act on applications for its own jobs, but cannot access or hire applicants for another employer's job.
8. Hiring one pending candidate marks the application Accepted, job Taken, and other pending applications Rejected atomically.
9. Logout clears the authenticated session; protected data is not served from cache after sign-out.
10. The landing slideshow advances every four seconds when not paused; manual controls work, reduced-motion disables automatic rotation, and navigation links remain consistent across pages.

7.3 Deployment requirements

1. Install PHP 8.1+ with `pdo_mysql`; install Composer dependencies with `composer install`.
2. Import `schema.sql` into MySQL. It creates/selects `aurora_group3`, creates users/jobs/applications tables, and applies guarded application-field migrations.
3. Configure `DB_HOST`, `DB_NAME`, `DB_USER`, and `DB_PASS` in the server environment.
4. Configure `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, and `SMTP_FROM` for sign-in OTP. Protect secrets and restart the PHP web server after environment changes.
5. Use HTTPS in production so session cookies are Secure; restrict access to logs and database backups; maintain a tested backup/restore process.

7.4 Traceability summary

Workflow	Requirements	Primary data entities
Registration / sign-in / OTP / logout	FR-03 to FR-07, FR-15; NFR-01, NFR-05	users; PHP session

Workflow	Requirements	Primary data entities
Browse / post jobs	FR-01, FR-02, FR-08; NFR-03	jobs, users
Apply / review / select candidate	FR-09 to FR-13; NFR-02, NFR-03	applications, jobs, users
All POST actions / output / errors	FR-14; NFR-04, NFR-06, NFR-08	CSRF session token; application logs

---

End of SkillsPassAurora Software Requirements Specification, Version 3.0.